

VotingEscrowLockPayoutActionEncod **Contract** *Aragon*

HALBORN

Prepared by:  HALBORN

Last Updated 01/28/2026

Date of Engagement: January 21st, 2026 - January 21st, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
1	0	0	0	0	1

TABLE OF CONTENTS

1. Introduction

2. Assessment summary

3. Test approach and methodology

4. Risk methodology

5. Scope

6. Assessment summary & findings overview

7. Findings & Tech Details

7.1 Votingescrow address can be unintentionally overwritten due to missing immutability check

1. INTRODUCTION

Aragon engaged **Halborn** to perform a security assessment of their smart contracts on January 21st, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The **VotingEscrowLockPayoutActionEncoder** contract in scope is a payout action encoder that converts token distributions into voting escrow lock creation calls, mapping each campaign to a specific VotingEscrow contract address.

2. ASSESSMENT SUMMARY

Halborn was allocated 1 day for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** did not identify any significant security risks; however, one enhancement was recommended to improve code clarity and maintainability, which was acknowledged by the **Aragon team**:

- **Add an immutability check to prevent accidental overwrites and clearly signal when a campaign is already configured.**

3. TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts.
- Fork testing against main networks.
- Static security analysis of scoped contracts, and imported functions (**Slither**).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY
(a) Repository: osx-capital-distributor (b) Assessed Commit ID: 6a40cfb (c) Items in scope: src/payoutActionEncoders/VotingEscrowLockPayoutActionEncoder.sol Out-of-Scope: Third-party libraries and economic attacks.
Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL0 HIGH0 MEDIUM0 LOW0 INFORMATIONAL1		
SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
VOTINGESCROW ADDRESS CAN BE UNINTENTIONALLY OVERWRITTEN DUE TO MISSING IMMUTABILITY CHECK	INFORMATIONAL	ACKNOWLEDGED - 01/27/2026

7. FINDINGS & TECH DETAILS

7.1 VOTINGESCROW ADDRESS CAN BE UNINTENTIONALLY OVERWRITTEN DUE TO MISSING IMMUTABILITY CHECK

// INFORMATIONAL

Description

The `setupCampaign()` function in `VotingEscrowLockPayoutActionEncoder` does not prevent overwriting an already-configured VotingEscrow address for a campaign. If an administrator inadvertently calls the function with an existing campaign ID, the VotingEscrow address is silently replaced, potentially fragmenting the token distribution across multiple contracts.

Code Location

 Copy Code

```
54 | function setupCampaign(uint256 _campaignId, bytes calldata _auxData) external override owner0rDao {
55 |     address votingEscrowAddress = decodeSetupCampaignParams(_auxData);
56 |     if (votingEscrowAddress == address(0)) {
57 |         revert ZeroAddressNotAllowed();
58 |     }
59 |     campaignVotingEscrow[_campaignId] = votingEscrowAddress; // @audit Silently overwrites existing value
60 |     emit CampaignVotingEscrowSet(_campaignId, votingEscrowAddress, msg.sender);
61 | }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (1.0)

Recommendation

Consider adding an immutability check to prevent accidental overwrites and provide clear feedback when a campaign has already been configured.

Remediation Comment

ACKNOWLEDGED: The Aragon team made a business decision to acknowledge this finding and not alter the contracts.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.